

Project File Structure

Why This Matters

The templates and guides in this bundle only help if they live somewhere sensible. A consistent folder structure means you (and any AI tool or person you work with) can always find the current version of a document, and can tell at a glance what stage the project is at.

This guide gives you one recommended structure. It is a starting point, not a rule. Use the parts that fit.

It Works Anywhere — GitHub Is Optional

The structure below is just **folders and Markdown files**. It works identically whether you keep it:

- in a **GitHub repository** (best for traceability),
- in a **cloud drive** (Google Drive, OneDrive, Dropbox), or
- in a **folder on your own computer**.

You do not need to be technical, and you do not need GitHub, to use this framework.

What GitHub *adds*, if you choose to use it later, is **version history** (a record of every change and who made it) and a natural **handoff point** (a “commit” marks a document as ready for review). Those are nice to have, not required. A dated folder on a cloud drive achieves much of the same discipline if you are consistent.

[!NOTE]

*If you are not using GitHub, the most important habit is simple: **do not keep multiple copies of the same document in different places**. One folder is the home for each document. Everything else is a link or a shortcut to it.*

The Recommended Structure

project-name/	
├─ README.md	← what this project is, its status, and where things
├─ 00-overview/	← the big picture
│ └─ master-brd.md	(Master BRD, or a short project brief)
├─ 01-requirements/	← what is needed
│ └─ frd.md	(small project: one FRD)
└─ modules/	(large project: one BRD per capability)

```

├── [module]-brd.md
├── requirements/                (optional: individual requirement records)
│   └── FR-[MOD]-001.md
├── 02-design/                  ← how it will be built
│   └── technical-design.md      (Technical Design Document)
├── 03-decisions/               ← why choices were made
│   └── DEC-001-[short-title].md
├── 04-issues/                  ← what is still open
│   └── issues-register.md
├── 05-handover/                ← passing the work on
│   └── handover.md
└── assets/                     ← diagrams, images, exported PDFs

```

Folder	What goes here	Template
README.md	A one-page summary: what the project is, its status, and links to the key documents.	(this bundle's README is an example)
00-overview	The vision and scope.	Master BRD
01-requirements	What the project must do and how well.	FRD (small) or Module BRDs (large); Requirement template
02-design	The technical approach.	Technical Design Document
03-decisions	A record of each material decision.	Decision Record
04-issues	Open questions and unresolved concerns.	Issues Register
05-handover	What someone needs to take over the work.	Handover
assets	Supporting files (images, diagrams, exports).	—

The numbers keep the folders in a logical order on screen and roughly follow the life of a project: overview → requirements → design → (build) → handover, with decisions and issues recorded throughout.

Small Project vs Large Project

You do not need every folder. Scale to the work (see the README's "Choosing Your Document Set").

Small / single-capability project — you may only need:

```

project-name/
├── README.md
├── 00-overview/master-brd.md    (or just a short brief)
├── 01-requirements/frd.md
└── 02-design/technical-design.md

```

Large / multi-capability project — use the `modules/` folder so each capability has its own BRD, and keep individual requirement records if the volume justifies it.

[!NOTE]

Do not create empty folders “just in case”. Add a folder when you have something to put in it. An empty `02-design` folder tells a reader the design exists somewhere; it doesn’t.

If There Is Code or a Website

This bundle is about the **documents** that govern a project. If your project also produces software or a website:

- Keep the **code in its own repository or folder** (for a website, that is usually a separate project entirely).
 - Keep these governance documents either in a `docs/` folder inside that repository, or in a separate documents folder that links to it.
 - The Handover and Technical Design documents are where you record *where the code lives* and *how to run it*.
-

Naming Things Consistently

Small habits that pay off as a project grows:

- **Folders and file names:** lowercase, words separated by hyphens — `technical-design.md`, not `Technical Design.md`. This avoids problems across different systems.
 - **Decisions:** `DEC-001-short-title.md`, numbered in order.
 - **Issues:** tracked in the issues register as `ISS-001`, `ISS-002`.
 - **Requirements:** use the IDs from the taxonomy guide — `FR-CRM-001`, `SR-SEC-003`.
 - **Dates:** always `YYYY-MM-DD` (e.g. `2026-06-09`). This sorts correctly and is unambiguous internationally.
-

The One Rule That Matters Most

Whatever structure you choose, be consistent and keep a **single home** for each document. Most of the chaos in AI-assisted projects comes not from a poor structure, but from the *same document existing in three slightly different versions* in three different places. Pick one home; link to it from everywhere else.

Version History

Version	Date	Author	Summary
0.1	2026-06-09	Claude Code	Initial draft
1.0	2026-06-13	Jason Masters	Initial Version (Approved)